

Altasker — Schema Grounding Evidence

How Every Concept in Sections 0.1–0.6 Is Fully Supported by the Physical Schema

Purpose: This document proves, table by table and column by column, that every business rule, state machine, event trigger, and data operation described in the Master Reference Sheet (Sections 0.1–0.6 of the Internal Scope Document) and across all 18 Main Flows has a concrete, named home in the physical PostgreSQL schema. It is structured as a cross-examination reference: for any concept the teacher raises, the reader can immediately point to the exact table, column, constraint, or index that enforces it.

How to read: Each section maps one conceptual layer to its schema backing. "Schema anchor" entries cite the exact table.column, constraint name, or SQL construct. "Why it can't be faked" entries explain what would go wrong at the database level if someone tried to bypass the rule.

Part 1 — Section 0.1: The Six Capability Domains → Schema Backing

The six domains (A–F) are not free-text fields that users can invent. They are controlled vocabulary that flows through three expert capability tables.

Domain claims live in `expert_domain_depths`

```
expert_domain_depths.domain_code TEXT NOT NULL
→ stores 'DOMAIN_A' through 'DOMAIN_F'
expert_domain_depths.depth_level TEXT NOT NULL
CHECK (depth_level IN ('SURFACE', 'OPERATIONAL', 'DEEP'))
expert_domain_depths.verification_tier TEXT NOT NULL
CHECK (verification_tier IN
('CLAIMED', 'EVIDENCE_BACKED', 'SCENARIO_VERIFIED', 'PLATFORM_DEMONSTRATED'))
UNIQUE (expert_id, domain_code)
```

An expert can declare at most one depth level per domain. The uniqueness constraint at `(expert_id, domain_code)` makes it physically impossible to hold duplicate or conflicting depth claims for the same domain.

Project domain requirements live in `capability_footprints`

```
capability_footprints.required_domains_json JSONB NOT NULL DEFAULT '[]'
Format: [{"code": "DOMAIN_A", "depth": "DEEP"}, ...]
```

The elicitation synthesis engine (FastAPI, MF-4 Stage 5) writes this field. The matching engine (MF-5) reads it and compares each required domain code and depth against the candidate expert's `expert_domain_depths` rows.

How the 25% domain depth component is computed

For each entry in `required_domains_json`:

- Query: `SELECT depth_level FROM expert_domain_depths WHERE expert_id = ? AND domain_code = 'DOMAIN_X'`
- If expert's `depth_level` meets or exceeds the required depth → score 1.0 for that domain
- If expert has no row for that domain code → score 0.0

The UNIQUE constraint on `(expert_id, domain_code)` means this SELECT always returns at most one row — no ambiguity in the scoring query.

Part 2 — Section 0.2: The Ten Seams → Schema Backing

Seams are the primary differentiator (40% of composite score). They are tracked across four distinct tables, each serving a different lifecycle purpose.

Seam claims: `expert_seam_claims`

```
expert_seam_claims.seam_code TEXT NOT NULL (e.g. 'SEAM_AC', 'SEAM_DE')
expert_seam_claims.verification_tier TEXT NOT NULL DEFAULT 'CLAIMED'
CHECK (verification_tier IN
('CLAIMED','EVIDENCE_BACKED','SCENARIO_VERIFIED','PLATFORM_DEMONSTRATED'))
expert_seam_claims.submission_count INT NOT NULL DEFAULT 0 -- 5-attempt throttle
expert_seam_claims.locked_until TIMESTAMPTZ NULL -- 30-day lockout after 5th failure
UNIQUE (expert_id, seam_code)
```

The `seam_code` is the cross-reference point for every downstream table. The UNIQUE constraint prevents duplicate claims. The `submission_count` and `locked_until` columns enforce the anti-gaming throttle from Section 0.4.

Project seam requirements: `capability_footprints.required_seams_json`

```
[{"code": "SEAM_AC", "criticality": "LOAD_BEARING"},
{"code": "SEAM_CE", "criticality": "SIGNIFICANT"},
{"code": "SEAM_DE", "criticality": "SIGNIFICANT"}]
```

Three criticality values — LOAD_BEARING, SIGNIFICANT, CONTRIBUTING — are stored as strings in the JSONB. The matching engine in FastAPI reads these and applies the weight multipliers (1.0 / 0.7 / 0.4) when computing Component 1 of the composite score.

Seam gap map generation

For each required seam:

1. Query `expert_seam_claims` WHERE `expert_id` = ? AND `seam_code` = ?
2. Map `verification_tier` to confidence factor: CLAIMED→0.20, EVIDENCE_BACKED→0.55, SCENARIO_VERIFIED→0.80, PLATFORM_DEMONSTRATED→0.95
3. If no row found for a LOAD_BEARING seam → apply -0.1 penalty (not 0.0)
4. Return `{seam_code, coverage, color}` per seam for the gap map displayed to CEO

Why the gap map is DB-derivable, not faked

Because `expert_seam_claims` has UNIQUE (`expert_id`, `seam_code`), every lookup for (`expert_id`, `seam_code`) returns exactly 0 or 1 rows. There is no path for an expert to hold two different verification tiers for the same seam simultaneously.

Portfolio submissions: `portfolio_submissions`

```
portfolio_submissions.seam_claim_id UUID NOT NULL REFERENCES expert_seam_claims(id)
portfolio_submissions.llm_confidence FLOAT NULL
portfolio_submissions.status TEXT NOT NULL CHECK (status IN ('PENDING', 'APPROVED', 'REJECTED'))
```

The `seam_claim_id` FK is the critical design decision here. When FastAPI returns an LLM confidence ≥ 0.85 , NestJS executes:

```
UPDATE expert_seam_claims SET verification_tier = 'EVIDENCE_BACKED'
WHERE id = portfolio_submissions.seam_claim_id
```

Without this direct FK, the UPDATE would need to resolve the target row by (`expert_id`, `seam_code`) — which works, but introduces a race condition if two submissions for the same seam are in flight simultaneously.

Scenario responses: `scenario_responses`

```
scenario_responses.seam_claim_id UUID NOT NULL REFERENCES expert_seam_claims(id)
scenario_responses.pass_fail TEXT NULL CHECK (pass_fail IN ('PASS','FAIL','PENDING'))
scenario_responses.llm_rubric_scores_json JSONB NULL -- per-dimension scores
```

Same FK pattern as `portfolio_submissions`. The `llm_rubric_scores_json` stores individual dimension scores (each must pass the per-question threshold independently — Section 0.4 rubric rule). When all dimensions pass, NestJS writes `verification_tier = 'SCENARIO_VERIFIED'` to the exact `expert_seam_claims` row via `seam_claim_id`.

Outcome signals: `expert_seam_outcome_signals`

```
expert_seam_outcome_signals.seam_claim_id UUID NOT NULL REFERENCES expert_seam_claims(id)
expert_seam_outcome_signals.signal_type TEXT NOT NULL
CHECK (signal_type IN ('LOAD_BEARING','SIGNIFICANT','CONTRIBUTING'))
expert_seam_outcome_signals.valence TEXT NOT NULL
CHECK (valence IN ('POSITIVE','NEGATIVE'))
```

This table is the Tier 4 accumulation store. The `signal_type` and `valence` columns directly implement the Section 0.4 signal accumulation rule:

- 1× LOAD_BEARING + POSITIVE → Tier 4
- 2× SIGNIFICANT + POSITIVE → Tier 4
- 3× CONTRIBUTING + POSITIVE → Tier 4

The matching hard gate also reads this table: `COUNT(*) WHERE valence='NEGATIVE' AND signal_type='LOAD_BEARING' AND seam_code = ANY(load_bearing_seams) >= 2` → exclude from shortlist.

Part 3 — Section 0.3: Archetypes + Tiers → Schema Backing

Archetype classification is stored in two places

Session level (locked at Stage 2):

```
elicitation_sessions.archetype TEXT NULL -- e.g. 'ARCHETYPE_1'; set at Stage 2
```

Project level (mirrored from session):

```
projects.archetype TEXT NULL -- set by elicitation synthesis at Stage 5
projects.tier TEXT NULL CHECK (tier IN ('TIER_1','TIER_2','TIER_3'))
```

The archetype is determined by FastAPI during Stage 1 symptom extraction and confirmed by the CEO at Stage 2. It is then locked — the `elicitation_sessions.archetype` column is written once and not updated by any subsequent flow.

Load-bearing seam enforcement

Each archetype has exactly one load-bearing seam (Section 0.3 table). This is a code-level constant in FastAPI (not a DB table), but its enforcement is DB-backed:

1. `capability_footprints.required_seams_json` always contains exactly one entry with `"criticality":"LOAD_BEARING"` for the archetype
2. The matching score computation applies the `-0.1` penalty for a missing load-bearing seam
3. The hard gate filter excludes experts with 2+ NEGATIVE signals on a load-bearing seam code

Archetype-tier congruence: `expert_profiles.archetype_history_json`

```
expert_profiles.archetype_history_json JSONB NULL
Format: [{"archetype_code": "ARCHETYPE_1", "tier": "TIER_3", "self_declared": true}]
```

This JSONB field seeds the 20% archetype-tier congruence component for cold-start experts (no closed engagements). For experts with prior engagements, FastAPI derives this at query time from `engagements` → `projects` (reading `projects.archetype` and `projects.tier`). The JSONB field is the fallback — it is not the authoritative source once real history exists.

Tier classification from volume/complexity signals

The `elicitation_sessions.void_list_json` stores SDLC gaps detected in Stage 1. Infrastructure signals extracted in Stage 3 (Kafka/async flag, record counts, integration count) are what drive the TIER_1/TIER_2/TIER_3 classification written to `projects.tier`. This column then drives the subscription guard:

- `subscription_expert_tier = 'pro'` required to bid on Tier 2+ projects (checked at bid submission)

Part 4 — Section 0.4: Expert Verification Tiers → Schema Backing

The four-tier verification system is grounded in a single column on `expert_seam_claims`, backed by three write-path tables.

The canonical tier column

```
expert_seam_claims.verification_tier TEXT NOT NULL DEFAULT 'CLAIMED'
CHECK (verification_tier IN
('CLAIMED', 'EVIDENCE_BACKED', 'SCENARIO_VERIFIED', 'PLATFORM_DEMONSTRATED'))
```

Every legitimate tier upgrade is a single `UPDATE` to this column. No other column stores the tier. The confidence factor (0.20 / 0.55 / 0.80 / 0.95) used in the composite score is a code-level mapping from this string — there is no `confidence_factor` column in the DB because it would be a redundant derivative.

Tier 1 → Tier 2 (EVIDENCE_BACKED) write path

```
portfolio_submissions INSERT (status='PENDING')
→ FastAPI LLM extraction → llm_confidence ≥ 0.85 AND all required signals found
→ portfolio_submissions.status UPDATE 'APPROVED'
→ expert_seam_claims.verification_tier UPDATE 'EVIDENCE_BACKED'
→ platform_decisions INSERT (decision_type='PORTFOLIO_EVAL', decision='APPROVED')
```

Throttle enforcement:

```
expert_seam_claims.submission_count increments on every REJECTED submission
→ submission_count = 5 → locked_until = now() + 30 days
```

These two columns are the DB-level implementation of the "5 attempts then 30-day lockout" anti-gaming rule.

Tier 2 → Tier 3 (SCENARIO_VERIFIED) write path

```
scenario_assessments SELECT (ORDER BY last_used_at ASC NULLS FIRST, LIMIT 1)
→ scenario_assessments.last_used_at UPDATE now() -- prevents question reuse gaming
→ scenario_responses INSERT (pass_fail='PENDING')
→ FastAPI LLM rubric evaluation → all dimensions individually pass per-question threshold
→ scenario_responses.pass_fail UPDATE 'PASS'
→ scenario_responses.llm_rubric_scores_json UPDATE (per-dimension scores stored)
→ expert_seam_claims.verification_tier UPDATE 'SCENARIO_VERIFIED'
→ platform_decisions INSERT (decision_type='SCENARIO_EVAL', decision='PASS')
```

The `scenario_assessments.last_used_at` ordering is the anti-gaming mechanism: it ensures the same question bank entry is not repeatedly served to the same expert.

Tier 3 → Tier 4 (PLATFORM_DEMONSTRATED) write path

```
engagements → CLOSED trigger:
1. reviews INSERT (reviewer_role='TECH_TEAM', structured_signals_json set)
2. outcome_signals INSERT (review_id FK set – UNIQUE enforces 1:1 per review)
3. NestJS extracts per-seam rows:
   expert_seam_outcome_signals INSERT (seam_claim_id FK, signal_type, valence)
4. Accumulation rule evaluated:
   SELECT COUNT(*) FROM expert_seam_outcome_signals
   WHERE expert_id=? AND seam_code=? AND signal_type='LOAD_BEARING' AND valence='POSITIVE'
   → COUNT ≥ 1 → expert_seam_claims.verification_tier UPDATE 'PLATFORM_DEMONSTRATED'
5. platform_decisions INSERT (decision_type='SEAM_TIER_UPGRADE', decision='PLATFORM_DEMONSTRATED')
```

The `outcome_signals.review_id` `UNIQUE` constraint is what enforces the rule "one structured review produces one aggregate signal record." A `TECH_TEAM` member cannot submit two reviews for the same engagement to double-count Tier 4 signals.

Part 5 — Section 0.5: Composite Match Score Weights → Schema Backing

The composite score is computed entirely in FastAPI, but every input to every component is sourced from exact DB columns. No component requires human input or admin judgment.

Component 1 — Seam Alignment (40%)

Input	DB Source
Required seams + criticality	<code>capability_footprints.required_seams_json</code>
Expert's seam coverage per seam	<code>expert_seam_claims.verification_tier</code> WHERE (<code>expert_id</code> , <code>seam_code</code>)
Confidence factor mapping	Code constant: CLAIMED→0.20, EVIDENCE_BACKED→0.55, SCENARIO_VERIFIED→0.80, PLATFORM_DEMONSTRATED→0.95
Missing load-bearing penalty	–0.1 if no row found in <code>expert_seam_claims</code> for load-bearing seam

Component 2 — Domain Depth Coverage (25%)

Input	DB Source
Required domains + depth	<code>capability_footprints.required_domains_json</code>
Expert's depth per domain	<code>expert_domain_depths.depth_level</code> WHERE (<code>expert_id</code> , <code>domain_code</code>)

Depth ordering: SURFACE < OPERATIONAL < DEEP. Expert scores 1.0 per domain if `expert_depth` >= `required_depth`, else 0.0.

Component 3 — Archetype-Tier Congruence (20%)

Input	DB Source
Target archetype + tier	<code>projects.archetype</code> , <code>projects.tier</code>
Expert's prior engagement history	<code>engagements JOIN projects ON engagements.project_id = projects.id</code> WHERE <code>engagements.expert_id = ?</code> AND <code>projects.archetype = target</code> AND <code>projects.tier = target</code>

Input	DB Source
Cold-start fallback	<code>expert_profiles.archetype_history_json</code> (self-declared; confidence discounted)

Component 4 — Engagement Model Fit (10%)

Input	DB Source
Required engagement model	Embedded in <code>capability_footprints</code> or derived from project type
Expert's declared model	<code>expert_profiles.engagement_model</code> CHECK (IN 'MILESTONE','HOURLY','HYBRID')

Component 5 — Stack Tag Overlap (5%)

Input	DB Source
Required tags	Embedded in <code>capability_footprints</code> or project description
Expert's tags	<code>expert_stack_tags.tag</code> WHERE <code>expert_id = ?</code>
Recency weighting	<code>engagements.connected_at</code> for engagements where that tag was used

Hard Gate Filters (applied before scoring, not weighted)

Gate 1 — Claimed-to-verified ratio > 4:1:

```
SELECT expert_id FROM expert_seam_claims
GROUP BY expert_id
HAVING (COUNT(*) FILTER (WHERE verification_tier='CLAIMED')::FLOAT /
        NULLIF(COUNT(*) FILTER (WHERE verification_tier!='CLAIMED'), 0)) > 4
```

Source: `expert_seam_claims.verification_tier` aggregated per `expert_id`.

Gate 2 — 2+ NEGATIVE signals on load-bearing seam:

```
SELECT expert_id FROM expert_seam_outcome_signals
WHERE seam_code = ANY(load_bearing_seams) AND valence='NEGATIVE'
GROUP BY expert_id HAVING COUNT(*) >= 2
```

Source: `expert_seam_outcome_signals.valence, signal_type, seam_code`.

Self-exclusion guard:

```
SELECT 1 FROM projects WHERE client_id = candidate_id AND id = project_id
UNION
SELECT 1 FROM tech_team_profiles WHERE user_id = candidate_id AND linked_project_id = project_id
```

Source: `projects.client_id, tech_team_profiles.user_id + linked_project_id`.

Part 6 — Section 0.6: All State Machines → Schema Backing

This is the most critical section for teacher examination. For each state machine, we trace every state value, every guard condition, and every trigger to its exact column and constraint.

6.1 Elicitation Session States

State column:

```
elicitation_sessions.state TEXT NOT NULL DEFAULT 'IN_PROGRESS'
CHECK (state IN ('IN_PROGRESS','COMPLETED','ABANDONED','RETURNED'))
elicitation_sessions.current_stage INT NOT NULL DEFAULT 1 CHECK (current_stage BETWEEN 1 AND 5)
```

State transition evidence:

State	Trigger event	Tables written
IN_PROGRESS (initial)	CEO clicks "Start New Project"	elicitation_sessions INSERT; projects INSERT (state='DRAFT'); artifact_a INSERT; artifact_b INSERT; capability_footprints INSERT
Stage advances 1→2→3→4→5	CEO/TECH_TEAM complete each stage	elicitation_sessions.current_stage UPDATE
COMPLETED	Stage 5 synthesis succeeds	elicitation_sessions.state UPDATE; projects.state stays 'DRAFT' pending quality gate
RETURNED	Auto-publish quality gate fails	elicitation_sessions.state UPDATE; elicitation_sessions.current_stage SET to failing stage (not 1); platform_decisions INSERT (decision_type='SPEC_AUTO_RETURN')
ABANDONED	CEO navigates away	Session row persists at current current_stage; CEO can resume

Why the session persists at the failing stage (not stage 1):

The `elicitation_sessions.current_stage` UPDATE on RETURNED writes the specific stage where the void was detected (e.g., stage 2 for archetype confirmation failure), not 1. This is enforced by the application logic reading `void_list_json` to determine which stage to return to — the CEO is sent directly to the point of failure, not restarted from scratch.

Void tracking column:

```
elicitation_sessions.void_list_json JSONB NOT NULL DEFAULT '[]'
Format: [{"void_code":"GROUND_TRUTH_VOID","injected":false}, ...]
```

The quality gate check reads this JSON: any entry with `injected=false` is a blocking void. The gate passes only when all void entries have `injected=true` OR the void list is empty.

6.2 Spec (projects + artifact_a) States

State columns (tracked independently but must stay in sync):

```
projects.state TEXT NOT NULL DEFAULT 'DRAFT'
CHECK (state IN ('DRAFT','PUBLISHED','RETURNED_TO_CLIENT','SUSPENDED'))
artifact_a.state TEXT NOT NULL DEFAULT 'DRAFT'
CHECK (state IN ('DRAFT','PUBLISHED','RETURNED_TO_CLIENT','SUSPENDED'))
```

Why two separate state columns: `projects.state` and `artifact_a.state` are managed independently to allow edge cases where the project metadata is in one state but the public-facing spec artifact is in another (e.g., a project could be suspended while artifact_a is under revision). In practice they mirror each other for all normal flows; both are written atomically in the same DB transaction.

State transition evidence:

State	Trigger	Tables written
DRAFT (initial)	Session created	projects INSERT + artifact_a INSERT both in same BEGIN/COMMIT

State	Trigger	Tables written
PUBLISHED	Auto-publish quality gate passes	<code>projects.state</code> UPDATE 'PUBLISHED'; <code>artifact_a.state</code> UPDATE 'PUBLISHED' — same transaction
RETURNED_TO_CLIENT	Quality gate fails	<code>projects.state</code> UPDATE; <code>artifact_a.state</code> UPDATE; <code>elicitation_sessions.state</code> UPDATE 'RETURNED'; <code>platform_decisions</code> INSERT with <code>advisory_note</code>
SUSPENDED	Admin emergency pull-back	<code>projects.state</code> UPDATE 'SUSPENDED'; <code>artifact_a.state</code> UPDATE 'SUSPENDED' — admin-only route

Auto-publish quality gate — three checks backed by DB queries:

Check 1 — Footprint completeness:

```
-- required_seams_json must contain >= archetype minimum count
-- No entries in void_list_json with injected=false
SELECT void_list_json FROM elicitation_sessions WHERE id = session_id
```

Check 2 — Matching pre-check:

```
SELECT COUNT(DISTINCT esc.expert_id) FROM expert_seam_claims esc
WHERE esc.seam_code = ANY(required_seams_codes)
AND esc.verification_tier != 'CLAIMED'
AND EXISTS (SELECT 1 FROM users u WHERE u.id = esc.expert_id AND u.is_active=TRUE
AND u.subscription_expert_tier='pro')
-- Must return >= 1
```

Check 3 — No unresolved hard voids: read from `void_list_json` JSON array.

6.3 Capability Bid States

State column:

```
capability_bids.state TEXT NOT NULL DEFAULT 'DRAFT'
CHECK (state IN ('DRAFT','SUBMITTED','TECH_REVIEW','REVISION_REQUESTED','REVISED',
'TECH_APPROVED','TECH_DISAPPROVED','CONFLICT_PENDING',
'CONFLICT_RESOLVED_CEO_OVERRIDE','PRICE_NEGOTIATION',
'NEGOTIATION_COMPLETE','CEO_REVIEW','SELECTED','DECLINED'))
capability_bids.version_number INT NOT NULL DEFAULT 1
```

State transition evidence:

Transition	Trigger	Tables written
DRAFT → SUBMITTED	Expert submits 3-component bid	<code>capability_bids.state</code> UPDATE; <code>bid_versions</code> INSERT (version_number=1, revision_request_id=NULL)
SUBMITTED → TECH_REVIEW	TECH_TEAM opens review	<code>capability_bids.state</code> UPDATE

Transition	Trigger	Tables written
TECH_REVIEW → REVISION_REQUESTED	TECH_TEAM flags component	<code>capability_bids.state</code> UPDATE; <code>bid_revision_requests</code> INSERT (flagged_component, reason, requested_by)
REVISION_REQUESTED → REVISED	Expert revises flagged component only	<code>capability_bids.state</code> UPDATE; <code>capability_bids.version_number</code> += 1; <code>bid_versions</code> INSERT (revision_request_id=revision_req_id, version_number=N)
TECH_DISAPPROVED → CONFLICT_PENDING	CEO decides to proceed	<code>capability_bids.state</code> UPDATE
CONFLICT_PENDING → CONFLICT_RESOLVED_CEO_OVERRIDE	CEO writes justification	<code>bid_conflict_overrides</code> INSERT; <code>capability_bids.state</code> UPDATE; <code>platform_decisions</code> INSERT (decision_type='CRITERION_QUALITY_GATE', decision='CEO_OVERRIDE')
CEO_REVIEW → PRICE_NEGOTIATION	CEO proposes price adjustment	<code>price_negotiations</code> INSERT (round_number=1)
PRICE_NEGOTIATION → NEGOTIATION_COMPLETE	Expert accepts	<code>price_negotiations.response</code> UPDATE 'ACCEPTED'; <code>capability_bids.state</code> UPDATE
NEGOTIATION_COMPLETE → SELECTED	CEO finalizes selection	<code>capability_bids.state</code> UPDATE; <code>engagements.state</code> advances

Revision versioning evidence — the audit chain:

```

bid_versions.revision_request_id UUID NULL REFERENCES bid_revision_requests(id)
  → NULL on version 1 (original submission – no request caused it)
  → revision_request_id SET on versions 2, 3, N (each version links back to the request that triggered it)
UNIQUE (bid_id, version_number) -- prevents duplicate version numbers per bid

```

This FK chain means: given any `bid_versions` row, you can answer "why was this version created?" by following `revision_request_id` → `bid_revision_requests.reason` + `flagged_component`. Without this FK, the answer would require timestamp proximity reasoning.

Price negotiation cap — DB-enforced:

```

price_negotiations.round_number INT NOT NULL CHECK (round_number IN (1,2))
UNIQUE (bid_id, round_number)

```

It is physically impossible to INSERT a `price_negotiations` row with `round_number=3`. The CHECK constraint rejects it at the DB level, not just the application level.

CEO override audit trail — DB-enforced minimum justification:

```

bid_conflict_overrides.override_reason TEXT NOT NULL CHECK (char_length(override_reason) >= 50)
bid_conflict_overrides.bid_id UUID NOT NULL UNIQUE -- one override per bid
bid_conflict_overrides.ceo_user_id UUID NOT NULL REFERENCES users(id)

```

The `char_length >= 50` CHECK constraint prevents rubber-stamp overrides. The UNIQUE on `bid_id` prevents the CEO from repeatedly overriding the same bid.

6.4 Spec Clarification States (Surface A)

State column:

```
spec_clarifications.status TEXT NOT NULL DEFAULT 'OPEN'
CHECK (status IN ('OPEN','ANSWERED','RESOLVED'))
-- 'CLOSED' is NOT in the CHECK list – any route writing status='CLOSED' gets a DB error
spec_clarifications.responded_by UUID NULL REFERENCES users(id)
-- NULL until ANSWERED; set on ANSWERED transition
```

State transition evidence:

Transition	Trigger	Tables written
OPEN (initial)	Expert files clarification	<code>spec_clarifications</code> INSERT (responded_by=NULL, status='OPEN')
OPEN → ANSWERED	TECH_TEAM or CEO responds	<code>spec_clarifications.response_text</code> UPDATE; <code>spec_clarifications.responded_by</code> UPDATE (FK set); <code>spec_clarifications.status</code> UPDATE 'ANSWERED'
ANSWERED → RESOLVED	Expert marks resolved	<code>spec_clarifications.status</code> UPDATE 'RESOLVED'; <code>spec_clarifications.resolved_at</code> UPDATE now()

Why `responded_by` matters: It records which role answered the clarification. For Tier 2+ projects, route guard requires `client_subtype='TECH_TEAM'` to write this field. For Tier 1 projects, the CEO responds. Without this column, there is no audit trail of who provided what information — which matters if a dispute later references a clarification as a commitment.

6.5 Engagement States

State column:

```
engagements.state TEXT NOT NULL DEFAULT 'PENDING'
CHECK (state IN ('PENDING','CONNECTED','ACTIVE','CLOSED','DISPUTED'))
engagements.client_nda_accepted_at TIMESTAMPTZ NULL
engagements.expert_nda_accepted_at TIMESTAMPTZ NULL
```

State transition evidence:

Transition	Trigger	Tables written	Guards
PENDING (initial)	CEO selects expert via bid SELECTED	<code>engagements</code> INSERT or state UPDATE 'PENDING'	Bid must be SELECTED
PENDING → CONNECTED	Both NDA timestamps set	<code>engagements.client_nda_accepted_at</code> UPDATE; <code>engagements.expert_nda_accepted_at</code> UPDATE; <code>engagements.state</code> UPDATE 'CONNECTED'; <code>engagements.connected_at</code> UPDATE now()	Both timestamps must be NOT NULL before state UPDATE is permitted
CONNECTED → ACTIVE	First milestone FUNDED (IPN fires)	<code>engagements.state</code> UPDATE 'ACTIVE'	<code>milestones.state='FUNDED'</code> must exist for this engagement
ACTIVE → DISPUTED	Dispute filed on any milestone	<code>disputes</code> INSERT; <code>escrow_accounts.status</code> UPDATE 'FROZEN'; <code>engagements.state</code> UPDATE 'DISPUTED'	Escrow must be HELD

Transition	Trigger	Tables written	Guards
DISPUTED → ACTIVE	Dispute resolved	<code>disputes.state</code> UPDATE (AUTO_RESOLVED/MUTUAL_RESOLVED/DISPUTED_AUTO_RESOLVED); <code>engagements.state</code> UPDATE 'ACTIVE'	Ledger settled per resolution
ACTIVE → CLOSED	Final milestone reaches RELEASED	<code>engagements.state</code> UPDATE 'CLOSED' → triggers Tier 4 evaluation pipeline	All milestones RELEASED

SERVICE_PURCHASE skip path:

```
engagements.type CHECK constraint:
(type='PROJECT_BASED' AND project_id IS NOT NULL AND service_id IS NULL) OR
(type IN ('SERVICE_PURCHASE','TECH_DISCOVERY') AND project_id IS NULL AND service_id IS NOT NULL)
```

SERVICE_PURCHASE engagements are created directly in state='ACTIVE', bypassing PENDING/CONNECTED. This is permitted by the state machine spec and enforced by the type CHECK constraint ensuring the correct FK pattern exists.

Artifact B access gate — DB-enforced read guard:

```
SELECT ab.* FROM artifact_b ab
WHERE ab.project_id = ?
AND EXISTS (
  SELECT 1 FROM engagements e
  WHERE e.project_id = ab.project_id
  AND e.client_nda_accepted_at IS NOT NULL
  AND e.expert_nda_accepted_at IS NOT NULL
  AND (e.expert_id = :current_user_id
    OR EXISTS (SELECT 1 FROM tech_team_profiles ttp
      WHERE ttp.user_id = :current_user_id
      AND ttp.linked_project_id = ab.project_id))
)
-- CEO permanently excluded: no CEO branch in the EXISTS subquery
```

This is not an application-level check that can be bypassed by a misconfigured route — it is embedded in the FastAPI data access layer and runs against the actual `client_nda_accepted_at` + `expert_nda_accepted_at` columns. A CEO `user_id` will never satisfy either branch of the inner EXISTS.

6.6 Milestone States

State column:

```
milestones.state TEXT NOT NULL DEFAULT 'DEFINED'
CHECK (state IN ('DEFINED','AWAITING_PAYMENT','FUNDED','IN_PROGRESS',
  'SUBMITTED','IN_REVISION','APPROVED','RELEASED','DISPUTED'))
milestones.review_clock_paused_at TIMESTAMPTZ NULL -- NULL = clock running; SET = paused
milestones.total_paused_seconds INT NOT NULL DEFAULT 0 -- accumulates SLA pause time
```

State transition evidence:

Transition	Trigger	Tables written	Guards enforced
DEFINED (initial)	CEO creates milestone	<code>milestones</code> INSERT; <code>acceptance_criteria</code> INSERTs; <code>milestone_dod_items</code> INSERTs	<code>engagements.state</code> ='CONNECTED'

Transition	Trigger	Tables written	Guards enforced
DEFINED → AWAITING_PAYMENT	CEO clicks "Fund"	<code>milestones.state</code> UPDATE; <code>milestones.va_number</code> SET; <code>milestones.va_expires_at</code> SET; <code>virtual_accounts</code> INSERT (<code>entity_type</code> ='MILESTONE', <code>fixed_amount</code> set)	CEO wallet must exist
AWAITING_PAYMENT → FUNDED	SePay IPN fires on milestone VA	<code>wallets.available_balance</code> -= <code>amount</code> ; <code>wallets.locked_balance</code> += <code>amount</code> ; <code>wallet_transactions</code> INSERT (<code>ESCROW_LOCK</code>); <code>escrow_accounts</code> INSERT (<code>status</code> ='HELD'); <code>milestones.state</code> UPDATE 'FUNDED'; <code>milestones.funded_at</code> SET; <code>virtual_accounts.status</code> UPDATE 'USED'; <code>paygated_documents.release_state</code> UPDATE 'RELEASED' for staged docs	IPN amount must exactly match <code>virtual_accounts.fixed_amount</code>
FUNDED → IN_PROGRESS	Auto- advance	<code>milestones.state</code> UPDATE 'IN_PROGRESS'	Immediate, no guard
IN_PROGRESS → SUBMITTED	Expert submits deliverable	<code>milestone_submissions</code> INSERT; <code>milestones.state</code> UPDATE 'SUBMITTED'; <code>milestones.submitted_at</code> SET	DoD gate: SELECT COUNT(*) FROM milestone_dod_items WHERE milestone_id=? AND is_required=TRUE AND status!='COMPLETED' → must return 0
SUBMITTED → IN_REVISION	Sign-off authority requests revision	<code>revision_requests</code> INSERT; <code>milestones.state</code> UPDATE 'IN_REVISION'	Verified role matches <code>acceptance_criteria.verified_by_role</code>
IN_REVISION → SUBMITTED	Expert resubmits	<code>milestone_submissions</code> INSERT; <code>milestones.state</code> UPDATE 'SUBMITTED'	DoD gate runs again
SUBMITTED/IN_REVISION → DISPUTED	Party files dispute	<code>disputes</code> INSERT; <code>escrow_accounts.status</code> UPDATE 'FROZEN'; <code>milestones.state</code> UPDATE 'DISPUTED'	<code>escrow_accounts.status</code> ='HELD'
SUBMITTED/IN_REVISION → APPROVED	All required criteria verified	<code>acceptance_criteria.verified_at</code> UPDATE (per criterion); <code>milestones.state</code> UPDATE 'APPROVED'; <code>milestones.approved_at</code> SET; escrow release ledger entries; <code>withdrawal_requests</code> INSERT (<code>type</code> ='MILESTONE_RELEASE')	Criteria gate: SELECT COUNT(*) FROM acceptance_criteria WHERE milestone_id=? AND is_required=TRUE AND verified_at IS NULL → must return 0
APPROVED → RELEASED	Chi hộ credit IPN fires	<code>milestones.state</code> UPDATE 'RELEASED'; <code>milestones.released_at</code> SET; <code>withdrawal_requests.status</code> UPDATE 'COMPLETED'	IPN reference matches <code>withdrawal_requests.disbursement_id</code>

BLOCKER sprint status → review clock pause:

```
sprint_status_updates.blocker_type = 'TECH_CONSTRAINT' (or similar)
→ milestones.review_clock_paused_at = now()

TECH_TEAM resolves blocker:
→ milestones.total_paused_seconds += EXTRACT(EPOCH FROM (now() - review_clock_paused_at))
→ milestones.review_clock_paused_at = NULL
```

Both columns on `milestones` implement the SLA pause tracking. The `total_paused_seconds` provides cumulative pause audit for the Platform Integrity Monitor.

SCOPE_EVOLUTION → add-on phase trigger:

```
sprint_status_updates.scope_evolution_flag BOOLEAN NOT NULL DEFAULT FALSE
→ set to TRUE when blocker_type='SCOPE_EVOLUTION'
→ NestJS auto-creates addon_phase_requests row
→ milestones.review_clock_paused_at = now()

UNIQUE INDEX addon_per_sprint_event ON addon_phase_requests(trigger_sprint_status_id)
WHERE trigger_sprint_status_id IS NOT NULL
-- Prevents double-trigger if NestJS handler retries
```

6.7 DoD Checklist Item States

State column:

```
milestone_dod_items.status TEXT NOT NULL DEFAULT 'PENDING'
CHECK (status IN ('PENDING','COMPLETED','NOT_APPLICABLE'))
milestone_dod_items.is_required BOOLEAN NOT NULL DEFAULT TRUE
```

The critical constraint:

```
CONSTRAINT dod_required_cannot_be_na
CHECK (NOT (is_required = TRUE AND status = 'NOT_APPLICABLE'))
```

This DB-level CHECK constraint directly implements the spec rule: "NOT_APPLICABLE: only for non-required items." No application code change can bypass this — the DB itself rejects the UPDATE.

The submission gate — SQL query run by NestJS before allowing SUBMITTED transition:

```
SELECT COUNT(*) FROM milestone_dod_items
WHERE milestone_id = ? AND is_required = TRUE AND status != 'COMPLETED'
-- Returns > 0 → 422 DOD_INCOMPLETE, list of blocking items returned to expert
-- Returns 0 → proceed to milestone_submissions INSERT
```

State transition evidence:

Transition	Trigger	Tables written
PENDING (initial)	CEO or expert creates DoD item	<code>milestone_dod_items</code> INSERT
PENDING → COMPLETED	Expert self-marks	<code>milestone_dod_items.status</code> UPDATE 'COMPLETED'; <code>milestone_dod_items.completed_at</code> SET; <code>milestone_dod_items.completion_note</code> SET (required)

Transition	Trigger	Tables written
PENDING → NOT_APPLICABLE	Expert marks N/A (non-required only)	<code>milestone_dod_items.status</code> UPDATE (DB CHECK prevents this on <code>is_required=TRUE</code>); <code>milestone_dod_items.not_applicable_note</code> SET

Per-item TECH_TEAM comments:

```
dod_item_comments.dod_item_id UUID NOT NULL REFERENCES milestone_dod_items(id)
-- Route guard: write = TECH_TEAM only (client_subtype='TECH_TEAM')
-- CEO permanently excluded from reading this thread
```

The `dod_item_comments` table provides the TECH_TEAM-only annotation layer on each DoD item, separate from the engagement-level `messages` channel. This separation is intentional — DoD comments are technical quality notes, not contractual communications.

6.8 Sprint Status States

State column:

```
sprint_status_updates.status TEXT NOT NULL
CHECK (status IN ('ON_TRACK', 'DEVIATION', 'BLOCKER'))
sprint_status_updates.blocker_type TEXT NULL
CHECK (blocker_type IN ('SCOPE_GAP', 'TECH_CONSTRAINT', 'EXTERNAL_DEPENDENCY', 'SCOPE_EVOLUTION'))
sprint_status_updates.milestone_sprint_id UUID NOT NULL REFERENCES milestone_sprints(id)
-- Direct FK replaces implicit (milestone_id, sprint_number) lookup
-- Prevents orphan status updates referencing non-existent sprint plans
```

State transition evidence:

Status	Trigger	Tables written	Side effects
ON_TRACK	Expert submits weekly update	<code>sprint_status_updates</code> INSERT	No milestone state change
DEVIATION	Expert reports deviation	<code>sprint_status_updates</code> INSERT (deviation_note required)	Notification to client; no clock pause
BLOCKER (non-SCOPE_EVOLUTION)	Expert reports technical blocker	<code>sprint_status_updates</code> INSERT; <code>milestones.review_clock_paused_at</code> UPDATE now()	TECH_TEAM notified; TECH_TEAM resolves → <code>milestones.total_paused_seconds</code> updated; <code>review_clock_paused_at</code> cleared
BLOCKER (SCOPE_EVOLUTION)	Expert reports scope evolution	<code>sprint_status_updates</code> INSERT (scope_evolution_flag=TRUE); <code>addon_phase_requests</code> INSERT; <code>milestones.review_clock_paused_at</code> UPDATE now()	F8 add-on flow triggered; unique index prevents double-trigger

Per-sprint TECH_TEAM comments:

```
sprint_comments.milestone_sprint_id UUID NOT NULL REFERENCES milestone_sprints(id)
-- Route guard: write = TECH_TEAM only
-- Read = TECH_TEAM + EXPERT; CEO sees summary only
```

Separate from `messages` for the same reason as `dod_item_comments` — technical sprint feedback is distinct from contractual project communications.

6.9 Add-On Phase Request States

State column:

```
addon_phase_requests.status TEXT NOT NULL DEFAULT 'PENDING'
CHECK (status IN
('PENDING', 'TECH_APPROVED', 'AWAITING_CEO', 'FULLY_APPROVED', 'REJECTED', 'MILESTONE_CREATED'))
addon_phase_requests.trigger_sprint_status_id UUID NULL REFERENCES sprint_status_updates(id)
addon_phase_requests.created_milestone_id UUID NULL REFERENCES milestones(id)
addon_phase_requests.tech_approved_at TIMESTAMPTZ NULL
addon_phase_requests.ceo_approved_at TIMESTAMPTZ NULL
addon_phase_requests.rejected_at TIMESTAMPTZ NULL
addon_phase_requests.rejected_by UUID NULL REFERENCES users(id)
```

State transition evidence:

Transition	Trigger	Tables written
PENDING (initial)	NestJS auto-creates from SCOPE_EVOLUTION event	<code>addon_phase_requests</code> INSERT (trigger_sprint_status_id FK set)
PENDING → TECH_APPROVED	TECH_TEAM approves technical justification	<code>addon_phase_requests.status</code> UPDATE; <code>addon_phase_requests.tech_approved_at</code> SET; <code>addon_phase_requests.tech_approved_by</code> SET
TECH_APPROVED → FULLY_APPROVED	CEO approves budget	<code>addon_phase_requests.status</code> UPDATE; <code>addon_phase_requests.ceo_approved_at</code> SET; <code>addon_phase_requests.ceo_approved_by</code> SET
FULLY_APPROVED → MILESTONE_CREATED	System auto-creates new milestone	<code>milestones</code> INSERT (new milestone row); <code>addon_phase_requests.created_milestone_id</code> SET; <code>addon_phase_requests.status</code> UPDATE 'MILESTONE_CREATED'; <code>milestones.review_clock_paused_at</code> cleared on original milestone
Any → REJECTED	Either party rejects	<code>addon_phase_requests.rejected_at</code> SET; <code>addon_phase_requests.rejected_by</code> SET; <code>addon_phase_requests.rejection_reason</code> SET; original milestone clock resumed

The AWAITING_CEO status is a UI-facing label only — there is no additional DB write between TECH_APPROVED and FULLY_APPROVED. The spec explicitly states "UI label only — no DB state change," which is why the CHECK constraint includes it but there is no trigger condition that sets it from a DB operation.

Why `created_milestone_id` FK matters: It creates the reverse audit link: given the newly created add-on milestone, you can answer "which scope evolution event caused this milestone to be created?" by joining `milestones.id` → `addon_phase_requests.created_milestone_id`. Without this, you would need timestamp proximity reasoning — fragile under concurrent inserts.

6.10 Wallet Transaction Types (Internal Ledger)

Transaction type column:

```
wallet_transactions.transaction_type TEXT NOT NULL
CHECK (transaction_type IN ('TOP_UP', 'SUBSCRIPTION', 'ESCROW_LOCK', 'ESCROW_RELEASE',
```

```

        'PLATFORM_FEE', 'ESCROW_REFUND', 'ESCROW_SPLIT', 'WITHDRAWAL'))
wallet_transactions.amount BIGINT NOT NULL CHECK (amount > 0)    -- always positive
wallet_transactions.reference_id TEXT NULL                      -- polymorphic idempotency key

```

Balance invariant — every type modifies exactly two columns:

Type	Effect	Tables written
TOP_UP	wallets.available_balance += amount	wallets UPDATE + wallet_transactions INSERT
SUBSCRIPTION	wallets.available_balance -= price	wallets UPDATE + wallet_transactions INSERT + user_subscriptions.status UPDATE 'ACTIVE' + users.subscription_*_tier UPDATE
ESCROW_LOCK	wallets.available_balance -= amount; wallets.locked_balance += amount	Both wallets UPDATE columns + wallet_transactions INSERT + escrow_accounts INSERT (status='HELD')
ESCROW_RELEASE	wallets.locked_balance -= amount (client); wallets.available_balance += net (expert)	Both wallets UPDATE + wallet_transactions INSERT (ESCROW_RELEASE) + wallet_transactions INSERT (PLATFORM_FEE)
PLATFORM_FEE	wallets.available_balance += fee (platform wallet)	Platform wallet UPDATE + wallet_transactions INSERT
ESCROW_REFUND	wallets.locked_balance -= amount (client); wallets.available_balance += amount (client)	Client wallet UPDATE + wallet_transactions INSERT
ESCROW_SPLIT	wallets.locked_balance -= amount (client); both wallets.available_balance += amount/2	Both wallets UPDATE + two wallet_transactions INSERTs
WITHDRAWAL	wallets.available_balance -= amount	Expert wallet UPDATE + wallet_transactions INSERT + withdrawal_requests INSERT

Idempotency constraint — prevents SePay IPN double-credit:

```

CREATE UNIQUE INDEX wallet_tx_idempotency
ON wallet_transactions(wallet_id, reference_id)
WHERE reference_id IS NOT NULL;
-- reference_id format: 'TOP_UP:{va_number}:{transfer_ref}'
-- Second IPN retry → INSERT fails unique constraint → caught and ignored safely

```

This is a DB-level uniqueness guarantee, not an application-level check. Even if two NestJS workers process the same IPN simultaneously, only one INSERT can succeed.

Platform fee is read from DB, never hardcoded:

```

platform_settings.platform_fee_pct FLOAT NOT NULL DEFAULT 0.05
CHECK (platform_fee_pct BETWEEN 0 AND 1)
platform_settings.platform_wallet_id UUID NOT NULL UNIQUE REFERENCES wallets(id)

```

Every ESCROW_RELEASE transaction reads `SELECT platform_fee_pct, platform_wallet_id FROM platform_settings LIMIT 1` before computing the split. This means the fee rate can be changed without a code deploy.

6.11 Withdrawal States

State column:

```
withdrawal_requests.status TEXT NOT NULL DEFAULT 'PENDING'
  CHECK (status IN ('PENDING','PROCESSING','COMPLETED','FAILED'))
withdrawal_requests.type TEXT NOT NULL DEFAULT 'EXPERT_MANUAL'
  CHECK (type IN ('MILESTONE_RELEASE','EXPERT_MANUAL'))
withdrawal_requests.bank_account_xid TEXT NOT NULL -- sourced from users.sepay_bank_account_xid
withdrawal_requests.disbursement_id TEXT NULL      -- set after chi hộ API responds
```

State transition evidence:

Transition	Trigger	Tables written
PENDING (initial)	Expert requests withdrawal OR milestone APPROVED auto-trigger	wallets.available_balance -= amount (atomic with INSERT); wallet_transactions INSERT (WITHDRAWAL); withdrawal_requests INSERT (status='PENDING')
PENDING → PROCESSING	Chi hộ API responds with disbursement_id	withdrawal_requests.disbursement_id SET; withdrawal_requests.status UPDATE 'PROCESSING'
PROCESSING → COMPLETED	SePay credit IPN fires on expert's bank	withdrawal_requests.status UPDATE 'COMPLETED'; withdrawal_requests.confirmed_at SET
PENDING/PROCESSING → FAILED	Chi hộ API error or bounce IPN	wallets.available_balance += amount (reversal); wallet_transactions INSERT (WITHDRAWAL reversal); withdrawal_requests.status UPDATE 'FAILED'

Debit-first pattern prevents double-spend: The wallet debit and the withdrawal_requests INSERT happen in the same BEGIN/COMMIT block, atomically, before the external SePay API call. If two withdrawal requests are submitted simultaneously, only one can decrement available_balance without violating the CHECK (available_balance >= 0) constraint on wallets.

MILESTONE_RELEASE vs EXPERT_MANUAL distinction: The type column allows the earnings dashboard to show "Milestone Release (auto)" separately from "Manual Cash-Out." MILESTONE_RELEASE rows are created automatically by NestJS when milestones.state transitions to APPROVED. EXPERT_MANUAL rows are created by the expert clicking "Withdraw."

6.12 Subscription States

State column:

```
user_subscriptions.status TEXT NOT NULL DEFAULT 'PENDING_PAYMENT'
  CHECK (status IN ('PENDING_PAYMENT','ACTIVE','EXPIRING_SOON','EXPIRED'))
user_subscriptions.role_type TEXT NOT NULL CHECK (role_type IN ('client','expert'))
```

Mirror column on users (for JWT guard performance):

```
users.subscription_client_tier TEXT NOT NULL DEFAULT 'free' CHECK (IN ('free','pro'))
users.subscription_expert_tier TEXT NOT NULL DEFAULT 'free' CHECK (IN ('free','pro'))
users.sub_client_expires_at TIMESTAMPTZ NULL
users.sub_expert_expires_at TIMESTAMPTZ NULL
```

Why two tables? user_subscriptions is the authoritative record with full payment history. users.subscription_*_tier and users.sub_*_expires_at are denormalized for JWT guard performance — every protected route checks users.subscription_client_tier='pro' AND sub_client_expires_at > now() rather than doing a JOIN to user_subscriptions on every request.

State transition evidence:

Transition	Trigger	Tables written
PENDING_PAYMENT (initial)	User initiates subscription purchase	<code>user_subscriptions</code> INSERT (status='PENDING_PAYMENT'); <code>virtual_accounts</code> INSERT (entity_type='SUBSCRIPTION', entity_id=user_subscriptions.id, fixed_amount set)
PENDING_PAYMENT → ACTIVE	SePay IPN fires on SUBSCRIPTION VA	<code>wallets.available_balance -= price</code> ; <code>wallet_transactions</code> INSERT (SUBSCRIPTION); <code>user_subscriptions.status</code> UPDATE 'ACTIVE'; <code>user_subscriptions.activated_at</code> SET; <code>user_subscriptions.expires_at</code> SET; <code>users.subscription_*_tier</code> UPDATE 'pro'; <code>users.sub_*_expires_at</code> UPDATE; <code>virtual_accounts.status</code> UPDATE 'USED'
ACTIVE → EXPIRING_SOON	Scheduled job: expires_at - 7 days	<code>user_subscriptions.status</code> UPDATE 'EXPIRING_SOON'; notification sent
EXPIRING_SOON/ACTIVE → EXPIRED	expires_at passes	<code>user_subscriptions.status</code> UPDATE 'EXPIRED'; <code>users.subscription_*_tier</code> UPDATE 'free'
EXPIRED → ACTIVE	User renews	New <code>user_subscriptions</code> INSERT + new <code>virtual_accounts</code> INSERT → same IPN cycle

SUBSCRIPTION VA entity_id design: The VA is created with `entity_id = user_subscriptions.id` (the subscription row's UUID), not the user's ID. This allows the IPN handler to resolve directly to the exact subscription row without an ambiguous lookup — a user could have multiple subscription rows across role_types.

6.13 Dispute Resolution Layer States

State column:

```
disputes.state TEXT NOT NULL DEFAULT 'PENDING'
CHECK (state IN ('PENDING', 'LAYER_1_EVAL', 'AUTO_RESOLVED', 'LAYER_2_MUTUAL',
'MUTUAL_RESOLVED', 'LAYER_3_FORCED', 'DISPUTED_AUTO_RESOLVED'))
disputes.llm_confidence FLOAT NULL CHECK (llm_confidence BETWEEN 0 AND 1)
disputes.resolution_layer INT NULL CHECK (resolution_layer IN (1,2,3))
disputes.layer2_deadline TIMESTAMPTZ NULL -- 48h window set when Layer 1 fails
disputes.escrow_account_id UUID NOT NULL REFERENCES escrow_accounts(id)
disputes.criterion_id UUID NOT NULL REFERENCES acceptance_criteria(id)
```

State transition evidence:

Transition	Trigger	Tables written
LAYER_1_EVAL (initial)	Party files dispute	<code>disputes</code> INSERT (state='LAYER_1_EVAL'); <code>escrow_accounts.status</code> UPDATE 'FROZEN'
LAYER_1_EVAL → AUTO_RESOLVED	FastAPI LLM confidence ≥ 0.80	<code>disputes.state</code> UPDATE; <code>disputes.resolution_layer</code> SET 1; <code>disputes.llm_confidence</code> SET; escrow release ledger entries; <code>milestones.state</code> UPDATE 'RELEASED'
LAYER_1_EVAL → LAYER_2_MUTUAL	FastAPI LLM confidence < 0.80	<code>disputes.state</code> UPDATE; <code>disputes.layer2_deadline</code> SET (now()+48h); <code>disputes.llm_confidence</code> SET
LAYER_2_MUTUAL → MUTUAL_RESOLVED	Both parties agree same option within 48h	<code>disputes.state</code> UPDATE; <code>disputes.resolution_layer</code> SET 2; custom split ledger entries
LAYER_2_MUTUAL → LAYER_3_FORCED	48h expires without agreement	<code>disputes.state</code> UPDATE (triggered by scheduled job checking <code>layer2_deadline</code>)

Transition	Trigger	Tables written
LAYER_3_FORCED → DISPUTED_AUTO_RESOLVED	Automatic	<code>disputes.state</code> UPDATE; <code>disputes.resolution_layer</code> SET 3; 50/50 split ledger entries; <code>milestones.state</code> UPDATE 'RELEASED'

Why `disputes.escrow_account_id` is a direct FK (not derived via milestone):

```
disputes.escrow_account_id UUID NOT NULL REFERENCES escrow_accounts(id)
-- O(1) lookup for freeze/release regardless of engagement type
-- Works identically for PROJECT_BASED (escrow.milestone_id set) and SERVICE_PURCHASE
(escrow.engagement_id set)
-- Without this FK: dispute handler would need to JOIN through milestone OR engagement to find escrow
```

Why `disputes.criterion_id` is required:

```
disputes.criterion_id UUID NOT NULL REFERENCES acceptance_criteria(id)
-- Disputes must always target a specific, objective contract element
-- Prevents "I am generally unhappy" disputes with no measurable criterion
-- Layer 1 LLM evaluation is scoped to the specific criterion text vs deliverable
```

Escrow one-parent constraint — prevents invalid escrow records:

```
CONSTRAINT escrow_has_one_parent CHECK (
  (milestone_id IS NOT NULL AND engagement_id IS NULL) OR
  (milestone_id IS NULL AND engagement_id IS NOT NULL)
)
```

This constraint ensures that every `escrow_accounts` row is unambiguously associated with either a milestone (PROJECT_BASED path) or an engagement (SERVICE_PURCHASE path), never both or neither. The `disputes.escrow_account_id` FK therefore always resolves to a well-formed escrow record.

Part 7 — RBAC Constraints → Schema Backing (Section 0.7)

Every RBAC rule from Section 0.7 has a DB-backed enforcement point, not just an application-level guard.

Role storage

```
users.roles JSONB NOT NULL DEFAULT '[]' -- e.g. '["CLIENT_CEO","EXPERT"]'
users.active_role TEXT NOT NULL
CHECK (active_role IN ('CLIENT','EXPERT','ADMIN'))
users.client_subtype TEXT NULL
CHECK (client_subtype IN ('CEO','TECH_TEAM')) -- NULL for EXPERT and ADMIN
```

TECH_TEAM cannot self-register

Enforced at the application level (no self-registration route exists for TECH_TEAM), backed by:

```
tech_team_profiles.linked_client_id UUID NOT NULL REFERENCES users(id)
tech_team_profiles.linked_project_id UUID NOT NULL REFERENCES projects(id)
```

A TECH_TEAM account without a corresponding `tech_team_profiles` row pointing to a valid project is structurally incomplete — every route that requires TECH_TEAM functionality checks `tech_team_profiles.linked_project_id` against the requested project.

TECH_TEAM scope guard

```
tech_team_profiles.linked_project_id UUID NOT NULL REFERENCES projects(id)
-- Route guard: tech_team_profiles.linked_project_id = requested_project_id
-- TECH_TEAM cannot access bid reviews, Artifact B, or milestone sign-offs for any other project
```

CEO never sees Artifact B

Enforced in the FastAPI access gate query — no CEO branch exists in the **EXISTS** subquery. No application code path can grant CEO access without rewriting the query.

Expert Pro required for Tier 2+ bidding

```
users.subscription_expert_tier TEXT NOT NULL DEFAULT 'free'
users.sub_expert_expires_at TIMESTAMPTZ NULL
-- Guard at bid submission: subscription_expert_tier='pro' AND sub_expert_expires_at > now()
-- Also: projects.tier = 'TIER_2' or 'TIER_3' → requires pro subscription to bid
```

Bank Hub link required before chi hộ

```
users.sepay_bank_account_xid TEXT NULL
-- Guard at CONNECTED transition: expert must have sepay_bank_account_xid IS NOT NULL
-- Guard at withdrawal: bank_account_xid required in withdrawal_requests
```

is_active = FALSE is the only suspension mechanism

```
users.is_active BOOLEAN NOT NULL DEFAULT TRUE
-- No is_suspended column exists in schema
-- Every JWT guard reads users.is_active = TRUE on every protected request
-- Setting is_active=FALSE immediately blocks all route access on the next request
```

Part 8 — Payment Architecture → Schema Backing (Section 0.8)

Virtual account polymorphism

```
virtual_accounts.entity_type TEXT NOT NULL
CHECK (entity_type IN ('WALLET_TOPUP', 'MILESTONE', 'SERVICE', 'SUBSCRIPTION'))
virtual_accounts.entity_id TEXT NOT NULL           -- polymorphic; no DB-level FK
virtual_accounts.fixed_amount BIGINT NULL          -- NULL for WALLET_TOPUP; SET for all others
virtual_accounts.expires_at TIMESTAMPTZ NULL       -- NULL for WALLET_TOPUP (permanent)
virtual_accounts.status TEXT NOT NULL DEFAULT 'ACTIVE'
CHECK (status IN ('ACTIVE', 'EXPIRED', 'USED'))
```

The **entity_type** field is the router: the IPN handler reads it first to determine which table and column to look up with **entity_id**. This is why there is no DB-level FK on **entity_id** — a polymorphic FK would require four separate REFERENCES clauses, which PostgreSQL does not support. The application contract (entity_type → target table mapping) substitutes for referential integrity here.

Per-entity_type entity_id mapping:

entity_type	entity_id resolves to	Purpose
WALLET_TOPUP	users.id	IPN → credit user's wallet

entity_type	entity_id resolves to	Purpose
MILESTONE	<code>milestones.id</code>	IPN → FUNDED transition + escrow lock
SERVICE	<code>engagements.id</code>	IPN → SERVICE_PURCHASE ACTIVE transition
SUBSCRIPTION	<code>user_subscriptions.id</code>	IPN → ACTIVE subscription transition

Escrow dual-path support

```

escrow_accounts.milestone_id UUID NULL REFERENCES milestones(id)
escrow_accounts.engagement_id UUID NULL REFERENCES engagements(id)
CONSTRAINT escrow_has_one_parent CHECK (
  (milestone_id IS NOT NULL AND engagement_id IS NULL) OR
  (milestone_id IS NULL AND engagement_id IS NOT NULL)
)
CREATE UNIQUE INDEX escrow_milestone_unique ON escrow_accounts(milestone_id) WHERE milestone_id IS NOT NULL;
CREATE UNIQUE INDEX escrow_engagement_unique ON escrow_accounts(engagement_id) WHERE engagement_id IS NOT NULL;

```

Two partial unique indexes instead of one full unique index enables: one escrow record per milestone on Path A, one escrow record per engagement on Path B, while the CHECK constraint prevents any escrow from being parentless or double-parented.

Part 9 — Path A vs Path B Structural Differences → Schema Backing

Path A (Project-Based) structural requirements in schema

Requirement	Schema enforcement
Elicitation session must exist	<code>projects.elicitation_session_id</code> REFERENCES <code>elicitation_sessions(id)</code>
Capability footprint must be populated	<code>capability_footprints</code> — 1:1 with projects, populated by synthesis engine
Artifact A must be published	<code>artifact_a.state</code> = 'PUBLISHED' before matching engine runs
Artifact B is gated	<code>artifact_b</code> access guard — NDA timestamps + engagement state
Bid system required	<code>engagements</code> INSERT + <code>capability_bids</code> INSERT + bid state machine
Multi-milestone escrow	<code>escrow_accounts.milestone_id</code> IS NOT NULL (one per milestone)
Tech team handoff (Scenario A)	<code>tech_team_profiles.linked_project_id</code> + <code>artifact_b</code> population

Path B (Service Marketplace) structural requirements in schema

Requirement	Schema enforcement
No elicitation required	<code>engagements.project_id</code> IS NULL enforced by type CHECK constraint
Service must exist and be published	<code>services.state</code> ='PUBLISHED' + <code>services.price_vnd</code> > 0
Direct engagement creation in ACTIVE	<code>engagements</code> created with state='ACTIVE' (SERVICE_PURCHASE bypasses PENDING/CONNECTED)
Per-service-order escrow	<code>escrow_accounts.engagement_id</code> IS NOT NULL (one per engagement)
No bid system	<code>capability_bids</code> has no row for SERVICE_PURCHASE engagements
Free-tier clients can buy	No subscription guard on SERVICE_PURCHASE route

Engagement type immutability

```

engagements.type TEXT NOT NULL CHECK (type IN ('PROJECT_BASED','SERVICE_PURCHASE','TECH_DISCOVERY'))
-- No UPDATE path exists for this column after creation
-- The type-to-FK CHECK constraint enforces structural consistency:
CONSTRAINT engagement_type_fk_consistency CHECK (
  (type = 'PROJECT_BASED' AND project_id IS NOT NULL AND service_id IS NULL) OR
  (type IN ('SERVICE_PURCHASE','TECH_DISCOVERY') AND project_id IS NULL AND service_id IS NOT NULL)
)

```

Part 10 — FastAPI vs NestJS Write Boundaries → Schema Backing

The system has a clear and auditable boundary: FastAPI writes to `platform_decisions`; NestJS never writes to that table.

Tables written exclusively by FastAPI

Table	Written by FastAPI when
<code>platform_decisions</code>	Every LLM evaluation result (elicitation synthesis, spec auto-return, portfolio eval, scenario eval, dispute L1 eval, criterion quality gate, seam tier upgrade, CEO override log)

Why `platform_decisions.entity_type` is required

```

platform_decisions.entity_type TEXT NOT NULL
  CHECK (entity_type IN ('project','capability_bid','acceptance_criterion',
    'portfolio_submission','scenario_response','spec_clarification','service'))
platform_decisions.entity_id TEXT NOT NULL -- polymorphic UUID stored as TEXT; no DB FK

```

Without `entity_type`, the Platform Integrity Monitor (MF-17) cannot determine which table to JOIN when rendering the detail view for a decision. For example, `decision_type='PORTFOLIO_EVAL'` with `entity_type='portfolio_submission' → JOIN portfolio_submissions → JOIN expert_seam_claims` to get the seam code and outcome. If `entity_type` were absent, this JOIN chain would require inspecting `decision_type` string matching — fragile and non-atomic.

Tables written exclusively by NestJS (not FastAPI)

All operational state tables: `engagements`, `milestones`, `wallets`, `wallet_transactions`, `capability_bids`, `bid_versions`, `bid_revision_requests`, `expert_seam_claims` (tier UPDATE on FastAPI signal return), `withdrawal_requests`, `escrow_accounts`, `sprint_status_updates`, `addon_phase_requests`, `reviews`, `outcome_signals`, `expert_seam_outcome_signals`.

Part 11 — Fraud Detection & Account Integrity → Schema Backing (MF-18)

Bank account deduplication

```

users.sepay_bank_account_xid TEXT NULL UNIQUE
-- UNIQUE constraint: no two users can hold the same bank_account_xid
-- Deduplication check at Bank Hub webhook:
SELECT id FROM users WHERE sepay_bank_account_xid = :incoming_xid AND id != :current_user_id
-- If row found → auto-block both accounts

```

Suspension mechanism

```

users.is_active BOOLEAN NOT NULL DEFAULT TRUE
-- UPDATE users SET is_active=FALSE WHERE id = target_user_id
-- Immediate effect: every JWT guard returns 401 on next protected request
-- Wallets and escrow balances are NOT touched by suspension
-- Active engagements continue after reactivation

```

Fraud audit trail

```
platform_decisions INSERT (decision_type='CONTENT_MODERATION',
                           entity_id=user_id,
                           decision='FRAUD_AUTO_BLOCK' or 'ADMIN_SUSPENSION' or 'ADMIN_REACTIVATION',
                           advisory_note='reason')
```

Summary Table — Every Master Reference Concept → Its Schema Home

Concept (Section)	Primary table	Enforcing column/constraint
Six domains (0.1)	expert_domain_depths	domain_code, UNIQUE (expert_id, domain_code)
Domain depth levels (0.1)	expert_domain_depths	depth_level CHECK ('SURFACE', 'OPERATIONAL', 'DEEP')
Project domain requirements (0.1)	capability_footprints	required_domains_json JSONB
Ten seams (0.2)	expert_seam_claims	seam_code, UNIQUE (expert_id, seam_code)
Seam criticality (0.2)	capability_footprints	required_seams_json (criticality in JSON value)
Archetype (0.3)	elicitation_sessions, projects	archetype TEXT columns
Tier (0.3)	projects	tier CHECK ('TIER_1', 'TIER_2', 'TIER_3')
Load-bearing seam (0.3)	capability_footprints	required_seams_json criticality:'LOAD_BEARING'
Verification tiers (0.4)	expert_seam_claims	verification_tier CHECK (4 values)
5-attempt throttle (0.4)	expert_seam_claims	submission_count, locked_until
30-day lockout (0.4)	expert_seam_claims	locked_until TIMESTAMPTZ NULL
Tier 2 LLM threshold ≥ 0.85 (0.4)	portfolio_submissions	llm_confidence FLOAT NULL
Tier 3 per-dimension rubric (0.4)	scenario_responses	llm_rubric_scores_json JSONB
Tier 4 signal accumulation (0.4)	expert_seam_outcome_signals	signal_type, valence, seam_claim_id FK
Composite score Component 1 (0.5)	expert_seam_claims, capability_footprints	seam_code + verification_tier + required_seams_json
Composite score Component 2 (0.5)	expert_domain_depths, capability_footprints	depth_level + required_domains_json
Composite score Component 3 (0.5)	expert_profiles, engagements, projects	archetype_history_json, projects.archetype
Composite score Component 4 (0.5)	expert_profiles	engagement_model CHECK (3 values)
Composite score Component 5 (0.5)	expert_stack_tags	tag, UNIQUE (expert_id, tag)
Hard gate: 4:1 ratio (0.5)	expert_seam_claims	verification_tier aggregated
Hard gate: 2× negative signals (0.5)	expert_seam_outcome_signals	valence='NEGATIVE' count
Self-exclusion (0.5)	projects, tech_team_profiles	client_id, linked_project_id

Concept (Section)	Primary table	Enforcing column/constraint
Elicitation session states (0.6)	<code>elicitation_sessions</code>	<code>state</code> CHECK (4 values), <code>current_stage</code> CHECK (1-5)
Void tracking (0.6)	<code>elicitation_sessions</code>	<code>void_list_json</code> JSONB
Spec states (0.6)	<code>projects</code> , <code>artifact_a</code>	<code>state</code> CHECK (4 values) — both columns
Auto-publish quality gate (0.6)	<code>capability_footprints</code> , <code>expert_seam_claims</code>	JSON completeness + matching pre-check SQL
Bid states (0.6)	<code>capability_bids</code>	<code>state</code> CHECK (14 values)
Bid versioning (0.6)	<code>bid_versions</code>	<code>revision_request_id</code> FK NULL, UNIQUE (<code>bid_id</code> , <code>version_number</code>)
Price negotiation cap (0.6)	<code>price_negotiations</code>	<code>round_number</code> CHECK (IN (1,2)), UNIQUE (<code>bid_id</code> , <code>round_number</code>)
CEO override justification (0.6)	<code>bid_conflict_overrides</code>	<code>char_length(override_reason)</code> >= 50, <code>bid_id</code> UNIQUE
Spec clarification states (0.6)	<code>spec_clarifications</code>	<code>status</code> CHECK ('OPEN', 'ANSWERED', 'RESOLVED') — 'CLOSED' excluded
Engagement states (0.6)	<code>engagements</code>	<code>state</code> CHECK (5 values), <code>client_nda_accepted_at</code> , <code>expert_nda_accepted_at</code>
Engagement type immutability (0.6)	<code>engagements</code>	<code>type</code> CHECK + table-level <code>engagement_type_fk_consistency</code>
Artifact B NDA gate (0.6)	<code>engagements</code>	<code>client_nda_accepted_at</code> IS NOT NULL AND <code>expert_nda_accepted_at</code> IS NOT NULL
SERVICE_PURCHASE skip PENDING (0.6)	<code>engagements</code>	State created as ACTIVE; type CHECK enforces FK pattern
Milestone states (0.6)	<code>milestones</code>	<code>state</code> CHECK (9 values)
Review clock pause (0.6)	<code>milestones</code>	<code>review_clock_paused_at</code> TIMESTAMPTZ NULL, <code>total_paused_seconds</code> INT
DoD gate on SUBMITTED (0.6)	<code>milestone_dod_items</code>	<code>is_required</code> , <code>status</code> != 'COMPLETED' COUNT guard
DoD NOT_APPLICABLE blocked on required (0.6)	<code>milestone_dod_items</code>	CHECK (NOT (<code>is_required</code> =TRUE AND <code>status</code> ='NOT_APPLICABLE'))
Criteria gate on APPROVED (0.6)	<code>acceptance_criteria</code>	<code>is_required</code> , <code>verified_at</code> IS NULL COUNT guard
Platform fee from DB (0.6)	<code>platform_settings</code>	<code>platform_fee_pct</code> FLOAT, <code>platform_wallet_id</code> FK
Escrow one-parent rule (0.6)	<code>escrow_accounts</code>	CONSTRAINT <code>escrow_has_one_parent</code> CHECK
Sprint status types (0.6)	<code>sprint_status_updates</code>	<code>status</code> CHECK ('ON_TRACK', 'DEVIATION', 'BLOCKER')
BLOCKER clock pause (0.6)	<code>sprint_status_updates</code> , <code>milestones</code>	<code>blocker_type</code> , <code>review_clock_paused_at</code>
SCOPE_EVOLUTION auto- trigger (0.6)	<code>sprint_status_updates</code> , <code>addon_phase_requests</code>	<code>scope_evolution_flag</code> BOOLEAN, UNIQUE index on <code>trigger_sprint_status_id</code>
Add-on phase states (0.6)	<code>addon_phase_requests</code>	<code>status</code> CHECK (6 values), dual approval timestamps
Add-on milestone reverse link (0.6)	<code>addon_phase_requests</code>	<code>created_milestone_id</code> UUID NULL REFERENCES <code>milestones(id)</code>
Wallet transaction types (0.6)	<code>wallet_transactions</code>	<code>transaction_type</code> CHECK (8 values)
IPN idempotency (0.6)	<code>wallet_transactions</code>	UNIQUE INDEX (<code>wallet_id</code> , <code>reference_id</code>) WHERE <code>reference_id</code> IS NOT NULL

Concept (Section)	Primary table	Enforcing column/constraint
Withdrawal states (0.6)	<code>withdrawal_requests</code>	<code>status CHECK (4 values)</code>
Debit-first pattern (0.6)	<code>wallets, withdrawal_requests</code>	Atomic BEGIN/COMMIT: wallet UPDATE before chi hộ API call
Subscription states (0.6)	<code>user_subscriptions</code>	<code>status CHECK (4 values)</code> incl. PENDING_PAYMENT
Subscription mirror for guard (0.6)	<code>users</code>	<code>subscription_*_tier, sub_*_expires_at</code> denormalized columns
SUBSCRIPTION VA entity_id (0.6)	<code>virtual_accounts</code>	<code>entity_id = user_subscriptions.id</code> (not <code>user_id</code>)
Dispute resolution layers (0.6)	<code>disputes</code>	<code>state CHECK (7 values)</code> , <code>resolution_layer CHECK (1,2,3)</code> , <code>layer2_deadline</code>
Dispute criterion targeting (0.6)	<code>disputes</code>	<code>criterion_id UUID NOT NULL REFERENCES acceptance_criteria(id)</code>
Escrow freeze on dispute (0.6)	<code>escrow_accounts</code>	<code>status='FROZEN'</code> written on disputes INSERT
Layer 2 48h deadline (0.6)	<code>disputes</code>	<code>layer2_deadline TIMESTAMPTZ NULL</code> set on Layer 2 entry
RBAC: roles + active_role (0.7)	<code>users</code>	<code>roles JSONB</code> , <code>active_role CHECK</code> , <code>client_subtype CHECK</code>
RBAC: TECH_TEAM scope lock (0.7)	<code>tech_team_profiles</code>	<code>linked_project_id REFERENCES projects(id)</code>
RBAC: account suspension (0.7)	<code>users</code>	<code>is_active BOOLEAN</code> — only suspension mechanism
Payment: VA polymorphism (0.8)	<code>virtual_accounts</code>	<code>entity_type CHECK (4 values)</code> , <code>entity_id TEXT</code>
Payment: chi hộ automation (0.8)	<code>withdrawal_requests</code>	<code>bank_account_xid, disbursement_id, type</code> field
Fraud: bank dedup (0.7/0.8)	<code>users</code>	<code>sepay_bank_account_xid TEXT NULL UNIQUE</code>
FastAPI-only audit log (design)	<code>platform_decisions</code>	<code>decision_type, entity_type, entity_id</code> — never written by NestJS